



The Open University

Mathematics and Computing: Level 2
M256 Software development with Java

M256

Index

The references for the entries in this index are given by the unit number as an emboldened numeral followed by the page number. For example, **1.19** refers to *Unit 1*, page 19.

A

abstract
 class **3.25**
 entity **3.18, 4.12**
 Abstract Windowing Toolkit (AWT) **13.6, 13.14, 13.59**
 abstraction **1.21, 1.28**
 acceptance
 test **2.18, 2.39, 10.8**
 test cases **13.50**
 testing **2.19, 13.50**
 access modifier **5.26, 7.51**
 accessibility **5.26**
 package **5.27, 7.52**
 private **5.27, 7.52**
 protected **5.27**
 public **5.27, 7.51**
 action button **11.38, 13.58**
 activation rectangle **1.19**
 adaptive method **1.39**
 aesthetics **12.18**
 affordability **5.5**
 affordance **11.23, 12.29**
 Agile Alliance **14.43**
 agile
 development process **1.41**
 method **14.43–44**
 alternative scenario **7.28, 10.23**
 analysis **1.29, 1.37**
 analyst **2.8–9, 2.39**
 animate agent **12.8**
 anthropomorphic language **6.45**
 anthropomorphism **6.45**

Apple

 Lisa **12.11**
 Macintosh **11.13, 11.18, 12.11**
 application **1.7**
 programming interface (API) **5.21**
 architecture **14.7**
 assertion **10.35**
 association **3.10, 3.12, 3.30, 4.8**
 candidate **4.16**
 derived **4.29**
 navigation **6.27**
 one-way **6.28, 8.20**
 recursive **4.20**
 two-way **6.28, 8.20, 8.26**
 attribute **3.18, 4.14**
 derived **4.29**
 value **3.10**
 attributes **3.10**
 aural cue **11.12**

B

back end **14.10**
 behaviour
 of an object **5.16**
 of a software system **1.16**
 behavioural
 design pattern **14.14**
 requirement **2.12–13, 2.18**
 black box testing **10.10, 13.50**
 block tag **10.18**
 border layout **13.42, 13.60**
 boundary **10.11**
 bucket **9.20**
 bug **10.9**

- business
 - analyst 2.7
 - area 2.7, 3.8
 - context (of a system) 2.18
 - domain tier 14.10
 - language 2.11
 - system 2.7
- button 11.38, 13.13
- C**
 - call (a method) 1.16
 - candidate
 - association 4.16
 - class 3.14, 4.7, 4.9
 - cascading 7.13
 - CASE (computer-aided software engineering) tools 1.49
 - check box 11.40, 13.58
 - child class 3.23
 - class 1.12, 3.10
 - abstract 3.25
 - candidate 3.14, 4.7, 4.9
 - child 3.23
 - conceptual 4.5
 - concrete 3.26
 - coordinating 6.12, 8.8, 12.39, 14.17
 - description 3.22, 4.5
 - diagram 3.10, 4.5
 - façade 14.14
 - flexibility 9.32
 - immutable 9.17
 - kinds of 3.17
 - member 5.26
 - notation for 3.17
 - parent 3.23
 - protocol 5.15
 - test 10.12
 - utility 9.15
 - client 1.17, 1.27, 2.9, 6.25, 13.9
 - client-server architecture 14.10
 - COBOL 5.9
 - code 2.9
 - duplication 9.33
 - code-based testing tool 1.50
 - coding tool 1.49
 - cognitive and evolutionary psychology 11.19, 12.8
 - cohesion 5.47, 7.15, 8.12
 - collaboration 1.16, 6.25, 12.21
 - collaborator 1.17
 - command button 11.38
 - command-line interface 11.18
 - commitment 12.30
 - Comparable 9.18
 - compareTo() 9.18
 - component 5.15, 6.9, 14.7, 14.29, 14.40
 - documentation 13.23
 - engineering 5.12
 - nesting of 13.23
 - non-visual 13.15
 - object-oriented 5.20, 14.7
 - protocol 5.20
 - replaceability 13.23
 - responsibility 5.47
 - reusable 13.23
 - visual 13.13, 13.58
 - component-based
 - approach 5.12
 - development 13.23
 - software 5.12
 - composition 5.42
 - computer
 - architecture 14.7
 - games 12.11
 - computer games console
 - Microsoft Xbox 12.11
 - Sony PlayStation 12.11
 - concept (user-oriented) 12.30

- conceptual
 - class 4.5
 - model 3.5, 3.64, 4.5, 4.7, 4.30, 6.5
- concrete class 3.26
- concurrency control 14.36
- concurrent access 14.11, 14.28
- consistency 12.13, 12.30, 12.34
- constraint 2.20, 2.39, 4.27
 - dynamic 4.28
 - real-world 4.28
 - static 4.28
 - on the system domain 3.7
- container 13.13, 13.58
- coordinating
 - class 6.12, 8.8, 12.39, 14.17
 - message 6.13, 8.9, 8.18, 10.23
 - method 6.13, 6.20, 7.7, 8.9, 8.11
 - method specification 6.22
 - object 5.32, 6.10, 6.12, 7.7, 8.9–10, 8.12, 8.18, 10.23, 14.17,
- coordination of behaviour 6.14
- copy constructor 5.39
- core
 - object 6.9–10
 - system 1.12, 3.7, 3.9, 5.24, 6.9, 6.46, 7.7, 8.8, 11.5, 11.32, 12.6, 12.25, 12.39
- coupling 5.47, 7.15
- creational design pattern 14.14
- cue
 - aural 11.12
 - haptic 11.12
 - visual 11.12

D

- data hiding 5.16
- decompiler 5.25
- default access modifier 5.27
- defensive copy 5.39
- delegate 5.43
- dependency 14.8
- deployment 14.28
- deployment diagram 14.29
- derived
 - association 3.60, 4.29, 7.36
 - attribute 3.57, 4.29, 7.31, 8.39, 8.41
- design 1.29, 1.37, 6.19
 - for all 12.19
 - for reuse 5.42
 - guideline 11.21
 - heuristic 12.30, 12.35
 - pattern 14.13
 - principle 7.12, 11.21, 12.13, 12.25, 12.29, 14.13
 - rationale 12.32
 - tool 1.49
 - with reuse 5.42
- designer 1.45, 2.9
- desktop metaphor 11.30, 12.13
- detailed design and implementation 1.29
- developer 12.6
- developing a conceptual model 1.29
- developing a user interface 1.29
- developing dynamic models 1.29
- dialogue (simple and minimal) 12.30
- dialogue box 11.43
- dialogue control 12.21
- direct manipulation 11.32, 12.13, 12.17
- distributed system 14.28
- doc
 - comment 10.18
 - form 1.48
- documentation 1.46, 12.23, 14.43
 - of components 13.23

drawing device (for child)
 Etchasketch 12.20–21
 Skedoodle 12.20
dynamic model 1.18, 6.7, 6.19, 6.25, 7.8,
 8.9, 10.6, 10.23
dynamic part (of implementation model) 9.5
Dynamic Systems Development Method
 (DSDM) 14.42

E

effectiveness 12.27
efficiency 5.7, 12.27
encapsulation 5.15
engineering component 5.12
enumerated type (enum) 9.12
Equaliser system 11.14
`equals()` 9.17
error message 12.30
error prevention 12.30, 12.32
Etchasketch 12.20–21
`EtchedBorder` 13.29
event 3.17, 4.12–13, 13.13, 13.59
 `componentShown` 13.40
 handler method 13.14, 13.44, 13.58
 handling code 13.14
 list selection 13.21
 value changed 13.21
exception 7.23
exits (clear marking of) 12.30
extraordinary HCI 11.14
eXtreme Programming (XP) 1.41, 2.11,
 14.43

F

façade
 class 14.14
 object 14.15
Façade pattern 14.14

feedback 11.24, 12.12, 12.29–30
figure of speech 4.10
first code iteration 10.14
fixture 10.9, 13.51
forking 7.13
Form Editor 13.15
forwarding method 9.29
frame 13.58
Free Software Foundation 14.45
front end 14.10
functionality (of a widget) 11.36

G

getter
 message 6.15
 method 5.36, 7.50
graphical user interface (GUI/GUI
 interface) 11.6, 11.18, 12.6
grid bag layout 13.42, 13.60
GROPE (force-feedback interface)
 11.12
grouping and ordering 12.30, 12.34
GUI widget 12.11
guidelines for rejection 3.15, 4.9

H

haptic cue 11.12
hashcode 9.20
`hashCode()` 9.20
hashing 9.20
Hawking, Stephen 11.14
HCI (human–computer interaction) 11.5,
 12.7
helper method 13.35
heuristic 12.30
 evaluation 12.31
Hospital System 2.5, 2.24, 2.39
 conceptual model 3.64

I

identifier 1.12

immutable

- class 9.17
- object 5.38, 9.17

implementation 1.29, 1.37, 5.19

- details 5.15
- model 6.6, 6.25, 7.5, 8.15, 9.5, 9.37, 10.5–6, 10.8, 10.14, 10.23

incremental

- prototyping 5.50
- software development 5.49

independence (of classes) 5.16

indirect users 12.19

inference (skill of) 12.8

information

- expert 7.13
- flow 12.37

inheritance 3.24, 5.42, 12.16

in-house standards 1.47

initial structural model 3.8, 3.64, 4.5, 6.5, 10.5

initial widget table 12.37,

Inspector window 13.15

instance 3.12

integrated development environment (IDE) 1.9

integration testing 10.12

integrity

- of a package 5.30
- of an object 5.16

interaction 1.16, 12.12

interaction design 11.17

interactive product 11.6, 12.6

interface 5.13

interface type 9.9

invalid scenario 7.22

invariant 3.47, 4.5, 4.23, 8.8, 8.29, 8.36, 8.39

invoke (a method) 1.16

iPod 11.11

ISO (International Organization for Standardization) 12.18

iteration 1.39

iterative method 1.39, 14.39, 14.44

J

JAR file 5.25

Java 11.18

- application 1.7

- Runtime Environment (JRE) 1.7

- Software Development Kit (SDK) 1.9

- types 9.12

Javadoc 1.48, 10.18, 10.21

Joint Application Development (JAD) 2.11

JUnit 1.50, 10.12–13, 10.34

L

label 11.44, 13.13, 13.58

language (user-oriented) 12.30, 12.34

language idiom 4.10

large-scale project 1.44

LaTeX 12.14–15

layer 14.9

layout manager 13.14, 13.58, 13.60

learnt affordance 11.26

legacy system 5.9, 14.17

lifeline 1.19

linguistics 11.19

link 1.13, 3.10, 3.30, 6.23

- creation 8.38

- one-way 6.28, 8.20

- removal 8.23

- two-way 6.28, 6.61, 6.64, 8.20, 8.22, 8.27, 8.35

Linux 14.45

Lisa 12.11

list 11.44, 13.58
list box 11.44
listener 13.14, 13.59
locking 14.37
logical architecture 14.7, 14.40

M

Macintosh 11.18, 12.11
maintainability 5.5, 5.7, 7.7, 7.15, 9.6
maintenance 1.29, 1.37, 5.7
memory load (of users) 12.30, 12.34
mental model 11.29, 12.13, 12.17
menu 11.36
 bar 11.36
 interface 11.19
 pull-down 11.36
message 1.16
metaphor 12.13
 desktop 11.30, 12.13
 direct manipulation 11.32
 physical manipulation 12.13
 physical space 12.13, 12.17
 user interface 11.28
method 1.37
 specification 5.22–23
Microsoft
 Windows 12.11
 Xbox 12.11
model 2.9
modelling language 1.33
model–view separation
 principle 14.11
MP3 player 11.11, 11.32
multiple releases 5.50
multiplicity 4.17
multi-user system 14.28
mutable object 5.38, 10.50

N

Name 9.15
NASA TLX (Task Load Index) tool 12.27
navigating an association 6.27
navigation 6.28, 8.15, 8.42
NetBeans 1.9, 14.45
NetBeans Guide 1.9
network architecture 14.7
Nielsen, Jakob 12.30
notation for classes 3.17
noun 4.7
noun phrase 4.7
null layout 13.42, 13.61

O

object 1.12, 3.10, 12.17
object diagram 1.12, 3.10–11, 3.30, 4.18
Object Management Group (OMG) 1.34, 5.45
object-oriented 1.27
 software component 5.20, 14.7
 technology and GUI interfaces 12.11, 12.16
one-way
 association 6.28, 8.20
 link 6.28, 8.20
open source software 14.44
open source development 14.44
operating systems 12.11, 12.19
orchestrating object 5.32
order of implementation 10.26, 10.40
ordering and grouping 12.30, 12.34
organisational unit 3.18, 4.12

P

package 5.21, 7.51
 accessibility 5.27, 7.52
 diagram 14.7

- pair programming 14.43
- palette 13.16
- panel 13.58
- paperless office 12.9
- parent class 3.23
- persistence 14.28, 14.30
- Person 9.34
- phase 1.28
- polymorphism 5.19, 12.16
- portability 5.7
- post-condition 5.23
- pre-condition 5.22, 6.50, 7.18, 7.46, 8.24, 8.33
- predictive method 1.39
- privacy leak 5.38
- private accessibility 5.27, 7.52
- program documentation 1.46
- programmer 1.45, 2.9
- project
 - documentation 1.46
 - failure 1.43
 - large-scale 1.44
 - management 1.44, 2.18
 - manager 1.45
- protected accessibility 5.27, 7.51
- protocol
 - of a class 5.15
 - of a component 5.20
- prototype 1.40, 2.11, 12.25, 14.39
- prototyping 5.50, 14.42
- public accessibility 5.27, 7.51
- Publish–Subscribe pattern 14.14, 14.25, 14.38
- pull-down menu 11.36

Q

- questionnaires 12.23

R

- radar 12.8
- radio button 11.40, 13.58
- rapid application development (RAD) 2.11, 14.42
- Rational Software Corporation 14.40
- Rational Unified Process (RUP) 14.40
- real-world
 - constraint 4.28
 - entity 3.10, 4.5, 4.7
- reasoning
 - by class 12.9
 - by inheritance 12.9
- recursive association 4.20
- refactoring 9.26, 13.35, 14.43
- regression testing 10.13
- rejection guidelines 3.15, 4.9
- reliability 5.5
- Remote Method Invocation (RMI) 14.30
- RemoteException 14.31
- requirements 1.7
 - analysis 2.13
 - analyst 1.45, 2.7
 - creep 2.23
 - document 2.7, 2.17, 2.32, 2.35, 4.5
 - elicitation 2.9
 - elicitation techniques 2.11
 - specification 1.29, 1.37, 2.5, 2.7
- responsibility
 - of a component 5.47
 - of an object 1.16, 5.16
- reusability 5.6, 5.10, 7.7, 7.15
- reuse 7.15, 9.6, 9.32
- review 1.40
- risk analysis 1.44
- role 3.17, 4.12

S

- safety-critical system 11.8
- scenario 6.19, 6.28, 7.8, 8.9, 8.17, 8.32, 10.23, 14.40
- scope 2.12
- screen 11.41, 12.31
- scroll pane 13.17, 13.59
- scrollable list 13.17
- seeing and pointing (rather than remembering and typing) 12.14
- sequence diagram 1.18, 2.9, 6.7, 6.18, 7.8, 8.9, 10.23
- serialisable (object) 14.30
- server 1.17, 6.25, 13.9
- `setListData()` 13.20
- setter method 5.36
- Shopping System 2.5, 2.32
- Simula 12.10
- singleton 14.22
- Singleton pattern 14.14, 14.17, 14.22
- situational analysis 12.24
- Skeddoodle 12.20
- Sketchpad 12.7
- slider 11.44
- Smalltalk 11.18, 12.10
- socio-technical system 11.16, 11.19, 12.21
- sociology 11.19
- software 1.8
 - component-based 5.12
 - developer 1.45
 - development 1.5
 - development method 1.37, 1.44
 - engineering 1.44, 5.49
 - model 1.30
 - open source 14.44
 - quality 1.44, 14.40
 - system 1.8
 - tester 1.45
 - software component
 - general 5.6, 5.12
 - object-oriented 5.20, 14.7
- Sony PlayStation 12.11
- specialisation 3.23
- specification
 - of a class's protocol 5.19
 - of a method 5.15, 5.22–23
 - of an engineering component 5.13
 - of an object-oriented software component 5.20
- stakeholder 2.9, 2.11
 - analyst 2.9
 - client 2.9
 - designer 2.9
 - programmer 2.9
 - user 2.9
- state 9.17
 - of a system 1.13
 - of an object 1.12
- static model 1.18
- stereotype 3.25
- storage tier 14.10
- storyboard 12.31
- strict UML diagrams 1.35
- structural design pattern 14.14
- structural model 3.65, 6.6, 9.5
- structured English 13.35
- stub 10.40
- subclass 3.23, 9.28
- substitutability 3.24
- superclass 3.23, 9.28
- Swing 13.6, 13.13, 13.58
- synonym 4.7
- system 1.8
 - code 1.8
 - domain 2.7, 2.18, 2.39, 3.5, 3.9, 4.5, 6.46
 - testing 10.13
- systems analyst 1.45, 2.7

T

tabbed screen (tabbed pane) **11.44, 12.36, 13.59**

tangible entity **3.17, 4.12**

target language **1.27**

task

- allocation **12.21**
- analysis **12.24**
- success rates **12.27**

technical writer **1.45**

techniques (for eliciting requirements) **2.11**

terminology (user-oriented) **12.30, 12.34**

test **10.8**

- acceptance **2.18, 2.39, 10.8**
- case **10.9, 10.12, 13.51**
- class **10.12**
- driver tool **1.50**
- method **10.12**
- runner **10.34**
- suite **10.13**

testability **5.7**

test-driven development **10.6, 10.9, 10.23**

testing **1.29, 1.37**

- acceptance **2.19, 13.50**
- black box **10.10**
- integration **10.12**
- regression **10.13**
- system **10.13**
- techniques **1.44**
- tool **1.50**
- unit **10.12**
- usability **12.26**
- white box **10.10**

text area **11.44, 13.13, 13.58**

text box **11.44**

text field **11.44, 13.58**

textual analysis **3.12, 4.7, 4.16**

think-aloud **12.27**

three-layered architecture **14.10**

three-tiered architecture **14.10**

tier **14.9**

tiered architecture **14.9**

timeboxing **14.41–42**

timeliness **5.5**

top-level container **13.13**

Torvalds, Linus **14.44**

`toString()` **9.22, 10.20**

two-way

- association **6.28, 8.20, 8.26**
- link **6.28, 6.61, 6.64, 8.20, 8.22, 8.27, 8.35**

typical scenario **10.23**

U

UML (Unified Modeling Language) **1.33, 14.40**

UML standard **1.34**

UML-type diagrams **1.35**

undo-ability **12.30**

unit testing **10.12**

universal access **12.19**

universal commands **12.15–16**

unmodifiable **5.41**

unnamed package **5.21**

usability **11.8, 12.29**

- heuristic **12.30**
- testing **12.26**

usage **14.8**

use case **2.13, 2.18–19, 2.39, 5.50, 10.6, 14.40**

use case-driven **10.13, 10.23**

usefulness **5.5**

- user 2.9
 - characterisation 12.24
 - needs analysis 11.20, 12.18
 - needs analysis document 12.24
 - observation 12.24
 - satisfaction 12.27
 - workload 12.27
- user interface 3.7, 6.9, 7.7, 8.8, 11.8, 11.16
 - designer 12.6
 - development 1.37
 - metaphor 11.28
 - of iPods 11.11
 - of search engines 11.9
 - related disaster 11.9
 - tier 14.10
- user-centred
 - approach 12.12, 12.25
 - design 11.19, 12.18, 12.35
- user-defined type 9.11
- utility class 9.15

V

- valid scenario 7.20
- validation 10.8

- value (of an attribute) 3.10
- verification 10.8
- virtual reality systems 11.12
- visibility 11.22, 12.29
- Visicalc (first spreadsheet application) 12.12
- visual
 - component 13.13
 - cue 11.12

W

- walk-through 6.24, 7.8, 8.10, 10.23
- waterfall method 1.38, 14.39
- white box testing 10.10
- widget 11.17, 13.13
- window 11.41, 13.13
- working scenario 8.5, 8.39
- World Wide Web 11.12
- WYSIWYG (what you see is what you get) 12.14

X

- Xerox PARC 11.18, 12.9
- Xerox Star 12.9, 12.13, 12.16, 12.29
- XP (eXtreme Programming) 1.41, 2.11, 14.43